Process Concept, Memory Layout, and PCB

A process is a program in execution. The memory layout of an active process consists of:
1. Text Segment: Compiled executable machine instructions.
2. Data & BSS Segments: Initialized and uninitialized global variables.
3. Heap: Dynamically allocated memory expanding upwards (malloc / new).
4. Stack: Local variables, return addresses, and activation records expanding downwards.

The Operating System maintains a Process Control Block (PCB) for each process containing: Process ID (PID), program counter, CPU registers, process state, CPU scheduling information, memory management tables, and open file descriptors.

Process State Transitions and Context Switching

A process transitions through lifecycle states: New -> Ready -> Running -> Waiting (Blocked) -> Terminated.

A Context Switch occurs when the CPU scheduler switches execution from process P1 to process P2:
1. The kernel saves P1's CPU state, registers, and program counter into P1's PCB.
2. The kernel loads P2's context from P2's PCB into hardware CPU registers.
3. Context switching incurs direct latency (register saving) and indirect latency (cache line invalidation and TLB flushing).

Threads, Concurrency, and Race Conditions

A thread is a basic unit of CPU utilization comprising a thread ID, program counter, register set, and stack. Multiple threads within the same process share the Text, Data, Heap, and file resources.

A Race Condition occurs when multiple threads concurrently access shared memory and the final result depends on the non-deterministic interleaving of instructions.
Critical Section Problem requires three conditions: Mutual Exclusion (only one thread in critical section), Progress (selection of waiting threads cannot be postponed indefinitely), and Bounded Waiting (a limit exists on attempts before entry).